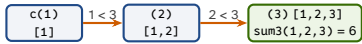


In one line: Functions are values: pass them, return them, let them close over state. Build behaviour from **small functions** over **values you do not mutate**: **currying** and **partial application** fix arguments, **pipe** chains steps, **toSorted** / **with copy** instead of mutating, **iterator helpers** pull one value at a time, and a **tagged template** is a function call on a template's pieces.

1 curry vs partial application

curry: one call per argument until `fn.length` are in



partial: fix some arguments now, the rest later



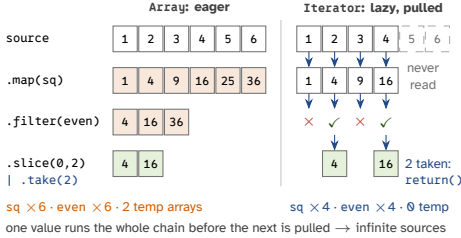
`add10.length = 3 - 1 = 2`. `.name = 'bound sum3'`

`fn.length` stops at the first default, skips rest:

`(a, b = 1, c) → 1 · (... xs) → 0 · ({a}, [b]) → 2`

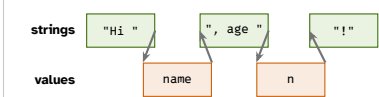
an auto-curry on those fires too early — pass the arity

2 eager arrays vs lazy iterators



3 tag`Hi \${name}, age \${n}!`

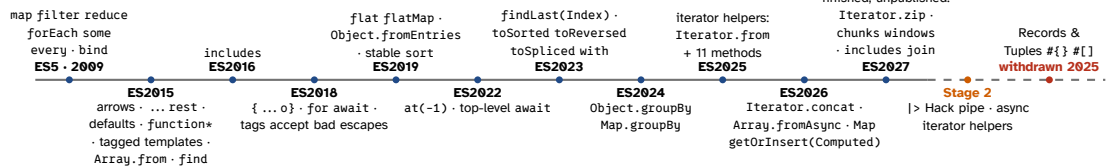
is the call `tag(strings, name, n)`:



`strings.length = values + 1` (an end may be "")
`strings + strings.raw`: frozen, and the
same array each time this call site runs
→ the tag can cache by its identity
values arrive as-is: not stringified, not escaped

4 When it landed

ESyyyy = the edition published that year;
Stage 4 = finished, waiting for its edition



Currying, partial application, composition

- Currying** turns `f(a, b, c)` into `f(a)(b)(c)`, one argument per call; an auto-curry (diagram 1) also takes several per step. **Partial application** fixes *some* and returns a function of the rest: `f.bind(t, 10), (... r) => f(10, ... r)`.
- `bind` also fixes this — ignored by arrows (lexical `this`) and by `new` (bound arguments are still prepended). The bound function has no prototype; `instanceof` uses the target's.
- `pipe(f, g)(x) = g(f(x))`: left to right, a reduce. `compose(f, g)(x) = f(g(x))`: right to left (maths order), a `reduceRight`. Steps take one value: `curry` first.
- pipe** is a **pipeline operator** — Stage 2, Hack style: the right side is any expression with a topic placeholder, `x |> f(x) |> g(x, 1)`; the `%` token is not final. TC39 has rejected F# pipes (right side is a unary function) several times.

```
const curry = (fn, n = fn.length) => function curried(...args) {  
  return args.length >= n ? fn.apply(this, args)  
    : (...more) => curried.apply(this, [...args, ...more]);  
};  
const sum3 = (a, b, c) => a + b + c;  
curry(sum3)(1)(2)(3); curry(sum3)(1, 2)(3); // 6, 6  
const pipe = (...fs) => x => fs.reduce((v, f) => f(v), x);  
const compose = (...fs) => x => fs.reduceRight((v, f) => f(v), x);  
const inc = x => x + 1, dbl = x => x * 2;  
pipe(inc, dbl)(3); compose(inc, dbl)(3); // 8, 7  
Array.from({ length: 4 }, (_, i) => i * i); // [0, 1, 4, 9]
```

Array methods — what comes back

method	returns	edge cases
<code>map(f)</code>	array, same length	holes stay holes; <code>f</code> is not called on them
<code>filter(p)</code>	array	keeps references, does not copy items
<code>flatMap(f)</code>	array	<code>map + flat(1)</code> ; a non-array stays one item
<code>reduce(f, init)</code>	any value	no <code>init</code> : starts at index 1, <code>[]</code> → <code>TypeError</code> , one item → returned, <code>f</code> never called
<code>find</code> , <code>findLast</code>	item	undefined if none — ambiguous if stored
<code>findIndex</code> , <code>findLastIndex</code>	index	— 1 if none
<code>some</code> , <code>every</code>	boolean	stop at the first decisive item; <code>[]</code> → <code>false</code> , <code>true</code>
<code>forEach(f)</code>	undefined	no break short of a throw
<code>Object.groupBy</code>	null-proto object	keys coerced to string or symbol
<code>Map.groupBy</code>	Map	any value as key

`groupBy` is static because `Array.prototype.groupBy` broke sites that load the Sugar library (versions until 14.0 patched it), and `.groupBy` broke code using arrays as hash maps.

Copy, don't mutate

mutates · returns	copies · returns the new array
<code>sort(cmp)</code> - the same array	<code>toSorted(cmp)</code>
<code>reverse()</code> - the same array	<code>toReversed()</code>
<code>splice(i, n, ... x)</code> - the removed items	<code>toSpliced(i, n, ... x)</code>
<code>a[i] = v · v</code>	<code>with(i, v)</code> — <code>RangeError</code> if out of range
<code>push(x)</code> , <code>unshift(x)</code> - new length	<code>[...a, x]</code> , <code>[x, ...a]</code>
<code>pop()</code> , <code>shift()</code> - the removed item	<code>a.slice(0, -1)</code> , <code>a.slice(1)</code>

- Copies are **shallow** and **dense**: holes become undefined (`[3, , 1].toSorted()` → `[1, 3, undefined]`); with counts a negative index from the end. Typed Arrays `get` to `Sorted`, `toReversed`, `with` — no `toSpliced`.
- Objects: `{ ... o, k: v }` copies one level. `Object.freeze(o)` returns the same `o` and is shallow — nested objects stay writable; writing a frozen property throws `TypeError` in strict code and is silently dropped in sloppy code.
- Records & Tuples** (deeply immutable, compared by value) were **withdrawn** at Stage 2 on 14 Apr 2025: no consensus for new primitives. The successor, **Composites**, makes objects, not primitives — early stage.

Lazy: generators + iterator helpers

- Calling a function* runs nothing: it returns a generator; each `next()` runs to the next `yield` and parks the frame — only that frame is kept, so a sequence can be infinite.
- Generators and the array, `Map`, `Set` and string iterators inherit `Iterator.prototype`, so helpers chain directly; an array needs `.values()`; any iterable or `next()` object → `Iterator.from(x)`.
- Lazy**, return an iterator: `map` `filter` `flatMap` take drop. **Consuming**: reduce to `Array` `forEach` `some` `every` `find`. `Iterator.flatMap` must get an iterator or iterable back — a string throws `TypeError`; `take`, `drop`: `RangeError` for a negative or NaN count.
- Single pass, no fork**: helpers share their source, and a consumed iterator just yields nothing — no error. `break` or a throw in `for...of` calls `return()`: the generator runs its finally and is finished for good.

```
function* naturals() { let n = 1; while (true) yield n++; }  
naturals().map(n => n * n).filter(n => n % 2 === 0)  
  .take(3).toArray(); // [4, 16, 36] - map ran 6 times  
Iterator.concat([1, 2], new Set([3])).toArray(); // [1, 2, 3]  
Iterator.zip([[1, 2, 3], ['a', 'b']]).toArray();  
// [[1, 'a'], [2, 'b']] shortest by default; 'longest', 'strict'  
Object.groupBy([1, 2, 3, 4], n => n % 2 ? 'odd' : 'even');  
// { odd: [1, 3], even: [2, 4] } null prototype  
const memo = f => { const cache = new Map(); // f(x) once per x  
  return x => cache.getOrInsertComputed(x, f); }  
const sql = (strs, ...vals) => {  
  text: strs.reduce((q, s, i) => q + '$' + i + s), values: vals };  
sql`SELECT * FROM t WHERE id = ${7}`;  
// { text: 'SELECT * FROM t WHERE id = $1', values: [7] }
```

Tags, IIFE, arguments, code from strings

- A **tagged template** may hold invalid escapes (`\unicode`): cooked `undefined`, raw kept; untagged it is a `SyntaxError`. `String.raw`Hi\n${2 + 3}!`` → `Hi\n5!` (6 characters). A tag may return anything — a function, a query object; an HTML tag must escape the values, an SQL tag sends them as parameters.
- IIFE** (function `() { ... }()`, `() => { ... }()`): the parentheses make it an expression — a private scope run once; each module now has its own. Modules have top-level `await`: an async IIFE is for CommonJS and classic scripts.
- arguments: array-like (no `map`), absent in arrows. In sloppy code with only simple parameters it is *aliased*: `arguments[0] = 99` changes `a`; strict code or default / rest / destructured parameters break the link. `... rest` is a real array.
- Direct `eval(s)` sees the local scope; `indirect(0, eval)(s)`, `eval?.(s)` and `new Function('a', 'return a + 1')` see only globals. node:vm: “not a security mechanism. Do not use it to run untrusted code.”

Traps

- `[1, 2, 3].map(parseInt)` → `[1, NaN, NaN]`: the index becomes the radix (`parseInt('2', 1)`). Use `map(Number)`.
- `[10, 9, 1].sort()` → `[1, 10, 9]`: compared as strings — and the array itself is reordered. `toSorted((a, b) => a - b)`.
- `xs.forEach(async x => ...)` returns before any `await` settles: use `for...of` + `await`, or `Promise.all(xs.map(f))`.
- A line starting with `(`, `[` or `{` continues the previous one: `const a = b + newline + ({}) => {}()` is `b((c) => {}())`. Start such a line with `;`.

Remember

Curry: one argument per call · partial: some now · pipe: left to right · copy: `toSorted`, with · iterators: lazy, one pass · tag: a call on the pieces.

Related: `js-scope-closures-this` (closures, `this`, `call` / `apply`) · `js-prototypes-objects-classes` (iterator protocol, generators as coroutines) · `js-async` (promises, `async` iteration) · `js-types-coercion-modules` (ESM, top-level `await`, copying values)